

Experiment – 2

Interfacing of LEDs with Arduino Uno

Date: 12/09/2026

Aim of the Experiment:

Interfacing of LEDs with Arduino Uno.

Objective:

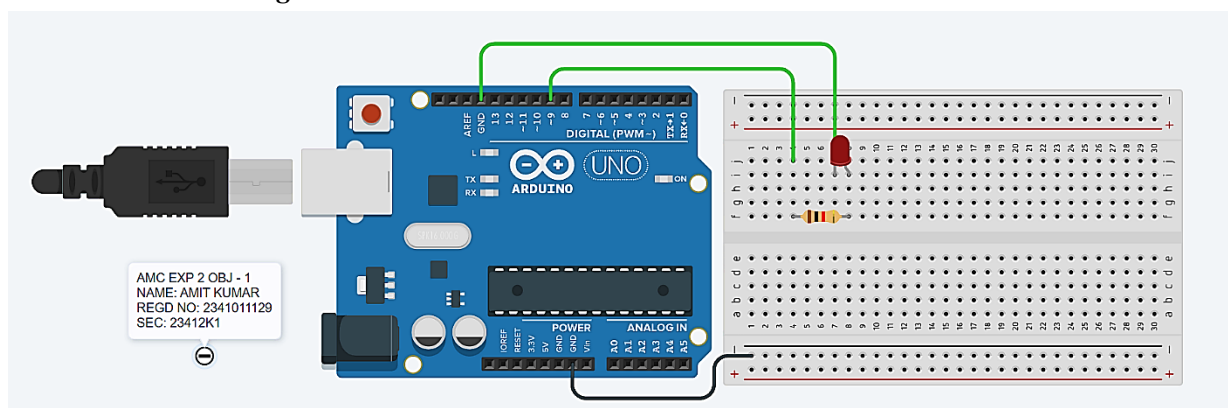
- 1) Create an Arduino Sketch to control the intensity of one or more LEDs.
- 2) SOS LED Signaling using Arduino
- 3) Design a Knight Rider using Arduino

Components/Equipment Required:

Sl No.	Name of the Component/Equipment	Specification	Quantity
1	Arduino UNO R3	16MHz	1
2	Arduino UNO cable	USB Type A to B	1
3	Resistors (carbon type)	¼ watt (330Ω) or 220 Ω	4
4	LED	Red, Blue, Green, White	As per the requirement
5	Breadboard	840 Tie points	1
6	Jumper Wire	-----	As per the requirement

Objective 1

Circuit/Schematic Diagram:



Experiment – 2

Interfacing of LEDs with Arduino Uno

Code:

```
int led = 9;
void setup()
{
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}
void loop()
{
  for(int i = 0; i <= 255; i++){
    analogWrite(led,i);
    Serial.println(i);
    delay(20);
  }
}
```

Observation: The LED intensity was successfully controlled using PWM. As the PWM value increased from 0 to 255, the LED brightness gradually increased from OFF to maximum brightness.

Objective 2

SOS LED Signalling using Arduino

Problem Statement:

Design and implement an **SOS LED Signaling System** using an **Arduino Uno**, one external LED, and a **current-limiting resistor**. The system should transmit the internationally recognized **SOS distress signal** in Morse code by controlling the LED with different **ON and OFF durations**.

The Morse code for SOS is represented as:

S → ••• (Three short flashes)

O → --- (Three long flashes)

S → ••• (Three short flashes)

Requirements

1. Connect an external LED to **Digital Pin 13** of the Arduino through a **220 Ω or 330 Ω current-limiting resistor**.
2. Configure Pin 13 as an **OUTPUT** in the setup() function.
3. In the loop() function, generate the SOS signal using the following timing:
 - **Dot (•):**
 - LED ON for 250 milliseconds
 - LED OFF for 250 milliseconds
 - **Dash (-):**
 - LED ON for 500 milliseconds

Experiment – 2

Interfacing of LEDs with Arduino Uno

- LED OFF for 500 milliseconds

Expected Output

The LED should continuously blink in the following pattern:

... — ...

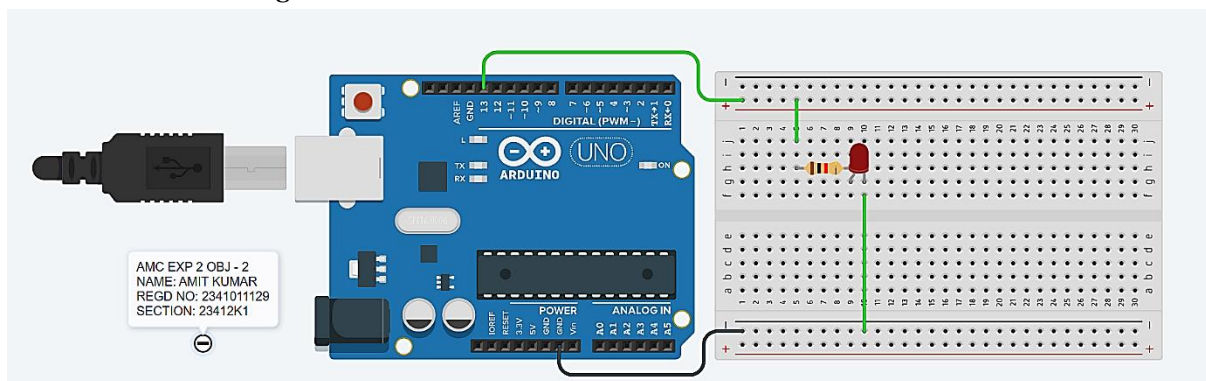
where:

- Short blink (Dot) = 250 ms ON, 250 ms OFF
- Long blink (Dash) = 500 ms ON, 500 ms OFF
- Pause between successive SOS signals = 3000 ms

Deliverables:

- Circuit diagram.
- Arduino program (.ino file).
- Demonstration of the working SOS LED signaling system.

Circuit/Schematic Diagram:



Experiment – 2

Interfacing of LEDs with Arduino Uno

Code:

```
void setup()
{
  pinMode(13, OUTPUT);
}

void dot()
{
  digitalWrite(13, HIGH);
  delay(200);
  digitalWrite(LED_BUILTIN, LOW);
  delay(200);
}

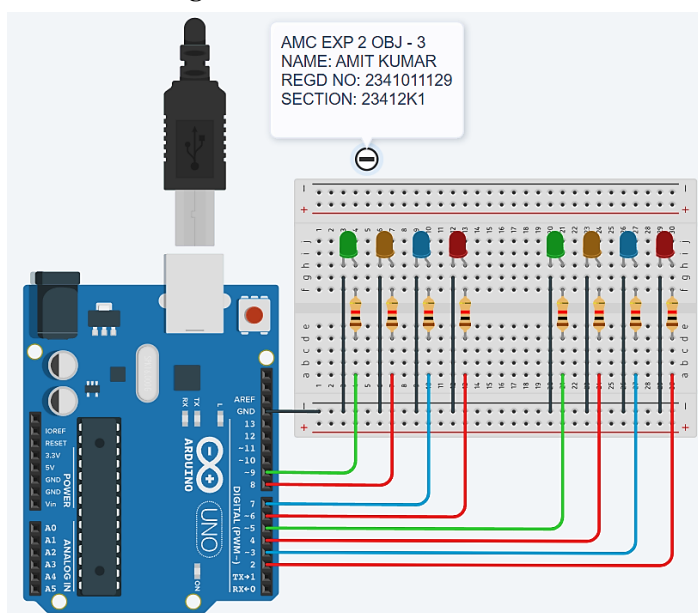
void dash() {
  digitalWrite(13, HIGH);
  delay(800);
  digitalWrite(13, LOW);
  delay(800);
}

void loop() {
  dot();
  dot();
  dot();
  delay(1500);
  dash();
  dash();
  dash();
  delay(1500);
  dot();
  dot();
  dot();
}
```

Observation: The external LED connected to Digital Pin 13 successfully transmitted the SOS distress sequence (· · · — — — · · ·) in Morse code. The system accurately demonstrated distinct timing intervals: three rapid short pulses (dots), followed by three longer sustained pulses (dashes), three short pulses, and an idle pause before repeating the cycle continuously

Objective 3

Circuit/Schematic Diagram:



Experiment – 2

Interfacing of LEDs with Arduino Uno

Code:

```
int count = 0;
int timer = 50;
const int LPins[] = {2,3,4,5,6,7,8,9};
void setup()
{
    for(int i = 0; i<8; i++){
        pinMode(LPins[i],OUTPUT);
    }
}

void loop()
{
    for(count = 0; count<8; count++){
        digitalWrite(LPins[count], HIGH);
        delay(timer);
        digitalWrite(LPins[count+1], HIGH);
        delay(timer);
        digitalWrite(LPins[count], LOW);
        delay(timer*2);
    }
    for(count = 8; count>0; count--){
        digitalWrite(LPins[count], HIGH);
        delay(timer);
        digitalWrite(LPins[count-1], HIGH);
        delay(timer);
        digitalWrite(LPins[count], LOW);
        delay(timer*2);
    }
}
```

Observation: The 8-LED array connected to Digital Pins 2 through 9 successfully demonstrated a sequential, bi-directional running light (chaser) effect. The LEDs illuminated and turned off in an overlapping forward sequence from pin 2 to pin 9 and immediately reversed direction back to pin 2, repeating continuously with a smooth scanning transition defined by the 50 ms delay timing.

Post Experiment Questionnaire:

- 1) List out the different types of Arduino boards available in the market and give one specific application of each board.

Solⁿ-

Arduino Board	Example Application
Arduino Uno	Basic robotics and LED projects
Arduino Nano	Compact wearable projects
Arduino Mega 2560	3D printers and complex robotics
Arduino Leonardo	USB keyboard/mouse emulation
Arduino Due	High-speed ARM-based applications
Arduino MKR WiFi	IoT and wireless projects

Experiment – 2

Interfacing of LEDs with Arduino Uno

2) Differentiate between pull-up and pull-down resistor.

Solⁿ-

Basis	Pull-Up Resistor	Pull-Down Resistor
Connection	Connected between input pin and VCC (+5V).	Connected between input pin and GND.
Default State	Keeps the input pin HIGH when no signal is applied.	Keeps the input pin LOW when no signal is applied.
Purpose	Prevents floating input and ensures a stable HIGH state.	Prevents floating input and ensures a stable LOW state.
Example	Push button connected to GND with a pull-up resistor.	Push button connected to VCC with a pull-down resistor.

3) When to use pull-up or pull-down resistors?

Solⁿ- Pull-up resistor: Used when the default input state should be HIGH, and becomes LOW when a switch is pressed.

Pull-down resistor: Used when the default input state should be LOW, and becomes HIGH when a switch is pressed.

4) What are PWM pins in Arduino? How many PWM pins are present in Arduino UNO?

Solⁿ- PWM (Pulse Width Modulation) pins generate variable-duty-cycle signals to simulate analog output, commonly used for controlling LED brightness and motor speed.

Arduino Uno has 6 PWM pins: 3, 5, 6, 9, 10, and 11.

5) Can we give power supply to the Arduino Uno through USB connection and DC power barrel jack simultaneously? Justify your answer.

Solⁿ- Yes, Arduino Uno can be connected to both USB and the DC barrel jack simultaneously. Its automatic power-selection circuit selects the appropriate power source and prevents direct conflict between supplies.

6) Which voltage regulator is present in Arduino module?

Solⁿ- Arduino Uno generally uses a 5V linear voltage regulator, commonly the NCP1117 (or compatible equivalent), to provide a stable 5V supply.

Name of the Student: Amit Kumar

Registration No: 2341011129

Branch, Section: CSE, 23412K1